Signal Minor Lab S7
Digital Communications Course

2nd year ENSEA

October 2026

The Signal Minor labs are carried out in Python. The notebook for this lab is available on Moodle. Answer the questions directly in this notebook. Attendance is mandatory and any unexcused absence will result in a grade reduction. Students who cannot attend for medical reasons must contact their supervising instructor promptly and follow the specific procedure they will be given.

You must validate 3 check-points per session, with your supervising instructor, who will grade them.

At the end of each session and before leaving the workstation, each pair is required to send their supervisor **the notebook containing the lab**: formulas and calculations (may be handed in on paper during the session), annotated plots (limited to 8 per session), with particular attention to the analysis and interpretation of the results.

Regarding the Python code: follow your supervising instructor's instructions.

Python Hint 1: Use `help (functionname)` *to get help on a function.*
Python Hint 2: to create the function `myfunction`*, use the syntax*

```
def myfunction(input1, input2, ...):
    return output1, output2
```

THEME 1: BPSK Baseband Communication

The three points to validate for Theme 1 are the following.

- ***Relationship between the signal-to-noise ratio (SNR) and the normalized SNR:*** $\frac{E_b}{N_0}$.
- ***Ideal channel: comparison between the bit error rate (simulator) and the theoretical curve.***
- ***Non-ideal channel: visualization of the BPSK constellation affected by ISI,*** $x_n,\ n = 0, \ldots, N-1$.

Remarks and tips

Notation: we will use the discrete-time signal notation of the Digital Communications module, where $x_n,\ n = 0, \ldots, N-1$ denotes the n-th sample.

Useful Python functions (via the numpy, matplotlib.pyplot, scipy, etc. libraries): `numpy.random.rand`, `numpy.random.randn`, `numpy.sign`, `scipy.signal.lfilter`, `matplotlib.pyplot.plot`, `xlabel`, `ylabel`, `title`, `scipy.special.erfc`, `numpy.where`, `len`, `matplotlib.pyplot.semilogy`, etc.

Hint: To plot the error rate we use the Monte Carlo method, i.e., the simulation of *a large number* N of binary transmissions. Among the N transmitted symbols, we compute the empirical frequency of error occurrences (comparison between a_n and $\widehat{a}_n$).

■

1 Ideal Nyquist AWGN channel

For simplicity, in this first part we will consider an ideal Nyquist channel with additive Gaussian noise (AWGN), illustrated in Figure 1. The objective is to study the bit error probability at the receiver.

♢ BPSK communication chain

• *Write a function* `genBPSK` *that takes* N *as input and outputs a vector of* N *samples of the BPSK modulated signal:* $a_n,\ n = 0, \ldots, N-1$. *You can use* `np.random.uniform` *to create a random sequence of values between* 0 *and* 1. *Then subtract* -0.5 *and take its sign using* `np.sign`[1].

• *Create a function* `genBruitN` *that takes a value* σ_b^2 *as input and outputs a sequence of* N *random noise samples* $b_n,\ n = 0, \ldots, N-1$ *following a Gaussian distribution with zero mean and variance* σ_b^2. *You can use the function* `np.random.normal`[2].

• *To implement the decision block, create a function* `decodeBPSK` *that takes as input a vector* r_n *of* N *real samples observed at the receiver and outputs a vector* $\widehat{a}_n$ *of* N *samples in* $\{-1, +1\}$, *which represent the estimates of the transmitted BPSK symbols (*$a_n,\ n = 0, \ldots, N-1$*).*

[1] To check that `genBPSK` behaves correctly, plot the histogram of its output.

[2] To check that `genBruitN` behaves correctly, plot the histogram of its output.

• *In the main program, simulate the communication chain for* $N = 512$, $\sigma_b^2 = 0.5$ *and* $g(0) = 1$. *Visualize the BPSK modulated signal,* a_n, *as well as the estimated signal,* $\hat{a}_n$. *What is the bit error rate at the receiver, i.e., the ratio between the number of erroneous symbols and the number of transmitted symbols* (N)*?*

♢ Bit error rate of the AWGN channel

• *Relate the signal-to-noise ratio SNR* $= \frac{g(0)^2}{\sigma_b^2}$ *to the normalized signal-to-noise ratio* $\frac{E_b}{N_0}$.

• *For each value* $\left[\frac{E_b}{N_0}\right]_{dB} = 0, \ldots 12$ ***[dB!]****, compute the noise variance* σ_b^2 *for* $g(0) = 1$ *as well as the corresponding bit error rate for* $N = 512$.

• *Plot the bit error rate curve as a function of* $\left[\frac{E_b}{N_0}\right]_{dB} = 0, \ldots, 12$ ***[dB!]*** *for* $N = 512$. *Comment.*

• *What is the minimum value of* N *needed to plot the bit error rate curve over the whole range of* $\left[\frac{E_b}{N_0}\right]_{dB}$*?*

♢ **Check-point 1** ***Relationship between the signal-to-noise ratio (SNR) and the normalized SNR:*** $\frac{E_b}{N_0}$.

♢ Theoretical bit error probability

• *Plot the theoretical bit error probability curve seen in class, as a function of the normalized signal-to-noise ratio* $\left[\frac{E_b}{N_0}\right]_{dB} = 0, \ldots, 12$ ***[dB!]****:*

$$P_{b(\min)}(e) = \mathcal{Q}\left(\sqrt{\frac{2E_b}{N_0}}\right).$$

You can use the function `sp.erfc`*, for example.*

♢ Comparison of the error curves: practical and theoretical

• *Compare the practical curve (obtained by simulation) with the theoretical curve. Comment.*

• *What SNR (in dB) guarantees a bit error probability of about* 10^{-3}*?*

♢ **Check-point 2** ***Ideal channel: comparison between the bit error rate (simulator) and the theoretical curve.***

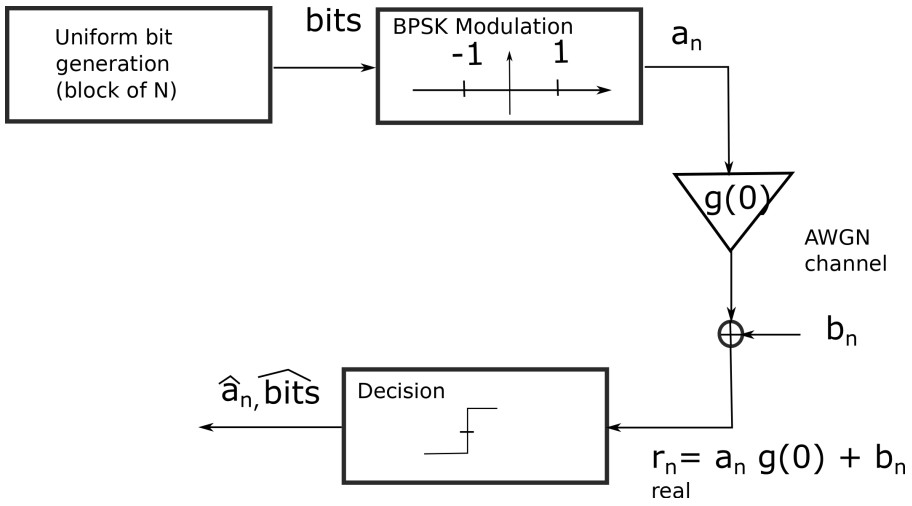


Figure 1: Baseband communication chain of a BPSK signal through an ideal Nyquist channel.

2 Intersymbol interference (ISI) and equalization

In this part, we will consider a convolutive channel that introduces ISI, as illustrated in Figure 2. We will study a simple equalization method, which makes it possible to reduce or cancel the effects of ISI at the receiver.

♢ Non-ideal channel

- *Create* `genSig` *that takes* N *as input and outputs a vector of* N *samples of the useful signal* $x_n,\ n = 0, \ldots, N-1$. *Use the function to filter the BPSK signal* $a_n,\ n = 0, \ldots, N-1$ *with* $G(z) = 0.5 + 0.2z^{-1}$ *in order to obtain the useful signal* $x_n,\ n = 0, ..., N-1$.
- *Write the expression of the impulse response of the overall channel filter* $g_n, \forall n$ *from the Z-transform* $G(z)$ *above. Then write the expression of* x_n *for* $n = 0, 1, 2, 3$, *assuming that* $a_n = 0,\ \forall n < 0$.
- *Visualize the useful signal* $x_n,\ n = 0, \ldots, N-1$. *Comment.*

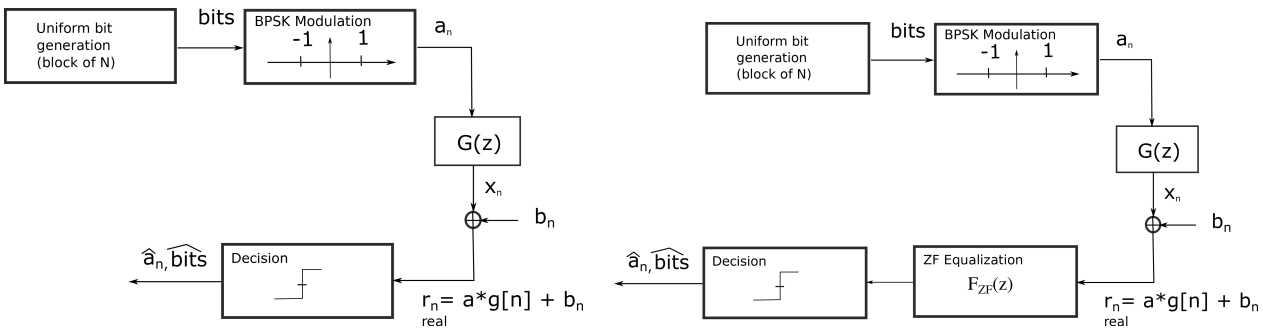
♢ **Check-point 3** ***Non-ideal channel: visualization of the BPSK constellation affected by ISI,*** $x_n,\ n = 0, \ldots, N-1$.

♢ Bit error rate of the non-ideal channel

- *Use the functions developed in part 1 to plot the bit error rate of the communication chain in Figure 2a as a function of* $\left[\frac{E_b}{N_0}\right]_{dB} = 0, \ldots, 12$. *What is the effect of ISI and of the non-ideal channel* $G(z)$*? Comment.*

♢ Equalization

- *A simple equalization method, called* zero forcing, *consists in inverting the filter* $G(z)$ *at the receiver. Create a function* `egaliZF` *that takes as input the sequence of* N *received symbols* $r_n,\ n = 0, \ldots, N-1$ *and outputs a sequence* $t_n,\ n = 0, \ldots, N-1$ *obtained by filtering the input with* $F_{ZF}(z) = G(z)^{-1}$.
- *Plot the bit error rate of the communication chain in Figure 2b as a function of* $\left[\frac{E_b}{N_0}\right]_{dB} = 0, \ldots, 12$. *Comment.*
- *What are the drawbacks of this method in practice?*



(a) figure Without equalization.

(b) figure With equalization.

Figure 2: Baseband communication chain of a BPSK signal through a non-ideal channel $G(z)$.

THEME 2: Equivalent Complex Baseband Communication of 16-QAM

The three points to validate for Theme 2 are the following.

- ***Visualization of the 16-QAM (complex constellation and Gray code).***
- ***Detection in the complex space of the constellation (test and visualization).***
- ***Comparison of the bit and symbol error curves (practical vs. theoretical).***

Remarks and tips

Notation: we will use the discrete-time signal notation of the Digital Communications module, where $x_n,\ n = 0, \ldots, N-1$ denotes the n-th sample.

Hint: To plot the bit error rate we use the Monte Carlo method, i.e., the simulation of *a large number* N of binary transmissions. Among the N transmitted bits, we compute the empirical frequency of error occurrences (comparison between bits_n and $\widehat{\mathrm{bits}}_n$). The symbol error rate is computed in a similar way by comparing the 16-QAM symbols a_n with their estimates $\widehat{a}_n$.

Python: You can look up the functions `int()` to convert a binary number to decimal, `bin()` to convert a decimal number to binary, `chr()` to get a character from an ASCII code, and `np.where()`, in addition to the functions used in the other Themes. ■

1 16-QAM constellation

The objective is to visualize the constellation and the Gray code of the 16-QAM, in a similar way to the QPSK constellation in Figure 3.

♢ Complex constellation and Gray code

• *Write a function* `const16QAM` *that returns a complex matrix,* mComplex *of size* 4×4*, containing all the possible values of the symbols* $a_n = a_n^I + j a_n^Q$ *with* $a_n^I, a_n^Q \in \{\pm 1, \pm 3\}$.

• *Write a function* `Gray16QAM` *that generates* mGray*, the matrix of size* 4×4 *containing the corresponding Gray codes (stored in decimal).*

QPSK example ($M = 4$)

The two matrices of size 2×2 of the QPSK (i.e., 4-QAM) illustrated in Figure 3 are:

$$\mathrm{mComplex} = \begin{pmatrix} -1+j & 1+j \\ -1-j & 1-j \end{pmatrix} \quad \text{and} \quad \mathrm{mGray} = \begin{pmatrix} 1 & 3 \\ 0 & 2 \end{pmatrix}, \quad \text{in binary} \quad \begin{pmatrix} '01' & '11' \\ '00' & '10' \end{pmatrix},$$

where mComplex is the matrix of complex symbols a_k and mGray is the matrix of Gray codes in decimal. *Warning:* the Gray mapping is not unique! ■

♢ Visualization of the 16-QAM

• *In the main program, generate the two matrices for the 16-QAM, initialize* $M = 16$ *and visualize the constellation points as well as the binary Gray labels using the code provided in the notebook.*

♢ **Check-point 1** ***Visualization of the 16-QAM (complex constellation and Gray code).***

2 Equivalent complex baseband transmission of 16-QAM

The objective here is to write a function that simulates the 16-QAM communication chain illustrated in Figure 4. To do this, each block will be written as a separate function, as detailed below.

♢ Transmitter

• *Write a function* `genBin` *that generates* bits_n, $n = 0, \ldots, N-1$, *a sequence of* N *uniformly distributed binary values in* $\{0, 1\}$, *using for example the function* `np.random.randint`.

• *Write a function* `mappingGray` *that encodes the binary sequence to be transmitted,* bits_n, *into a sequence of* $N/4$ *16-QAM symbols* a_n, *using the two matrices* mComplex *and* mGray *determined in part 1. Explain why the symbol sequence* a_n *has a length equal to* $N/4$.

QPSK example ($M = 4$)

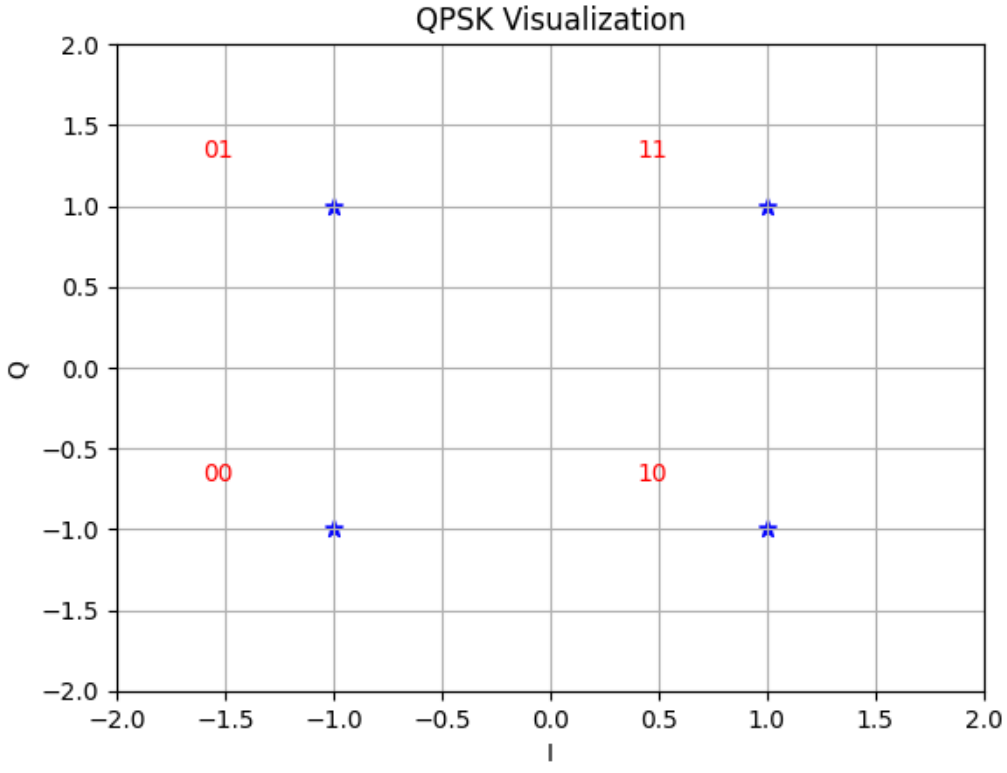


Figure 3: Visualization of the QPSK (complex constellation and Gray code).

For a QPSK, suppose that the binary sequence to be transmitted ($\text{bits}_n,\ n = 0, \ldots, N$) is:
$0,\ 1,\ 1,\ 0,\ 0,\ 1,\ 0,\ 0,\ 1,\ 1,\ 1,\ 1,\ \ldots$
• process the bits in groups of two (the group size is $\log_2 M$, where M is the size of the constellation)
$0,\ 1 \mid 1,\ 0 \mid 0,\ 1 \mid 0,\ 0 \mid 1,\ 1 \mid 1,\ 1 \mid \ldots$
• convert them to decimal
$1, \quad 2, \quad 1, \quad 0, \quad 3, \quad 3, \ldots$
• use the matrix mGray from Example 1 to identify the position (row, col) of each code
$(1,1);\ (2,2);\ (1,1);\ (2,1);\ (1,2);\ (1,2);\ \ldots$
• then read $\text{mComplex}(\text{row}, \text{col})$ (Example 1) to identify the corresponding sequence of QPSK symbols
$a_n,\ n = 0, \ldots, N$:
$-1+j,\ 1-j,\ -1+j,\ -1-j,\ 1+j,\ 1+j,\ \ldots$

■

Remarks

Unlike BPSK (Theme 2), for which the Gray mapping is trivial: $0 \to -1$ and $1 \to 1$; when $M > 2$ the bit generation block must be explicitly included in the communication chain. The binary information to be transmitted is processed in packets of size $\log_2(M)$, which are encoded into complex symbols of the M-QAM. ■

♢ Complex AWGN channel

• *Write a function* `canalAWGN` *that takes as input the complex sequence of* $N/4$ *symbols* a_n *and the noise variance* σ_b^2*, and outputs the sequence of* $N/4$ *complex values* r_n *at the receiver input. The additive noise* b_n *is a complex Gaussian variable with independent real and imaginary parts such that* $b_k^I, b_k^Q \sim \mathcal{N}\left(0, \sigma_b^2\right)$*. For simplicity, we choose* $g(0) = 1$*.*

• *In the main program, test all these functions by visualizing the received symbols* r_n *on the same figure as the 16-QAM symbols for* $N = 512$ *and* $\sigma_b^2 = 0.5$*.*

♢ Receiver

• *The decision block takes as input the sequence of* $N/4$ *complex values* r_n *and returns the sequence of* $N/4$ *estimated symbols* $\widehat{a}_n$ *of the 16-QAM constellation. You can use a threshold comparison method (similar to BPSK) in the 2D complex space. However, this method is tedious and the resulting code is complex. As an alternative, you can also look for, for each observation* r_n*, the closest complex symbol of the 16-QAM. Test your method by visualizing a single received symbol together with the corresponding estimated symbol.*

• *Write a function* `demapGray` *that performs the inverse operation of the function* `mappingGray`*. This function will use the two matrices* mComplex *and* mGray *to reconstruct the binary sequence* $\widehat{\text{bits}}_n$ *of length* N *from the* $N/4$ *estimated symbols* $\widehat{a}_n$*.*

♢ **Check-point 2** ***Detection in the complex space of the constellation (test and visualization).***

♢ Overall transmission chain

• *Write an overall function* `chaine16QAM` *that simulates the transmission chain of Figure 4. This function takes as input two values* N, σ_b^2 *and* $g(0)$*; as output it returns four vectors: two vectors* bits_n *and* $\widehat{\mathrm{bits}}_n$ *of length* N *and two vectors* a_n *and* $\widehat{a}_n$ *of length* $N/4$*. Test this function.*

3 Detection error curves

♢ Bit and symbol error rates

• *For an M-QAM: look in the course slides for the relationship between the* $\mathrm{SNR} = \frac{g(0)}{\sigma_b^2}$ *and the normalized SNR,* $\frac{E_b}{N_0}$.

• *For each value of* $\left[\frac{E_b}{N_0}\right]_{dB} = 0, \ldots, 12$ *[dB!] compute* σ_b^2 *for* $g(0) = 1$*, as well as the corresponding symbol and bit error rates of your simulator* `chaine16QAM` *from part 2. For the symbol error rate, you must count the number of symbols that differ between the transmitted sequence* a_n *and the estimated sequence* $\widehat{a}_n$*. For the bit error rate, you must count the number of bits that differ between the transmitted sequence* bits_n *and the estimated sequence* $\widehat{\mathrm{bits}}_n$.

♢ Bit and symbol error probabilities (theoretical)

• *Write the formula of the bit error probability* $P_{b(min)}(e)$ *parameterized by* M*, the size of the constellation. Plot the bit error probability curve for* $M = 16$.

• *Write the relationship between the symbol error probability and* $P_{b(min)}(e)$ *above for Gray coding, assuming sufficiently small noise. Plot the symbol error probability curve for* $M = 16$.

♢ Analysis and discussion

• *Compare the curves estimated with the simulator and the theoretical curves (for bit and symbol errors). Comment.*

• *Compare the baseband communication of BPSK and the passband communication of 16-QAM in terms of bit rate, bit error probability and spectral efficiency. Comment.*

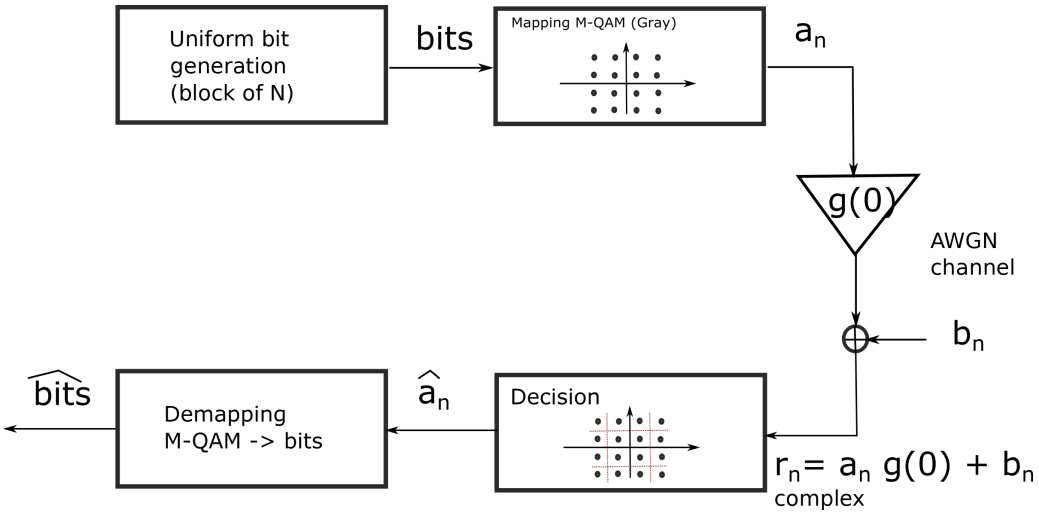


Figure 4: 16-QAM communication chain.

• *If time permits, also plot the error curves of the QPSK (by adapting the simulator to the case $M = 4$) and make a complete comparison: BPSK vs. QPSK vs. 16-QAM.*

♢ **Check-point 3** ***Comparison of the bit and symbol error curves (practical vs. theoretical).***